

CSE411 Computer Vision

Project Task 5

Implementation - Part 2 | Approximately 50% Overall Implementation

Project Title	Driver Drowsiness Detection System
Team Number	25
Team Members	Vangara Sridhar - 2023BCS0106 Suragouni Sahithi - 2023BCS0070 Vatte Ramya Sri - 2023BCS0208 Abhay Devanand Sawale - 2023BCS0046
Continuation	Continues Project Task 4 - Implementation Part 1
Date	25 September 2026

Task 5 focus: continue the proposed methodology, integrate the modules implemented so far, document the observed results, and identify the remaining modules.

1. Task 5 Objective and Progress

Project Task 4 established dataset collection, preprocessing, model training structure, webcam capture, backend inference, and the first alarm logic. For Task 5, these modules have been integrated into a runnable full-stack pipeline and connected to a current drowsiness-trained checkpoint.

Approximately 50% implementation status

Work area	Task 4 baseline	Task 5 integrated state
Data pipeline	Dataset configured and split.	UTA-RLDD mapping, class preparation, and YOLO training outputs available.
Model	Model interface and inference service.	Current checkpoint loaded once at startup with class-name validation.
Application	Individual frontend/backend modules.	Webcam -> API -> inference -> result display is integrated and tested.
Alarm	Conceptual/initial temporal logic.	Five-second continuous Drowsy timer, audio-file hook, and tone fallback.
Integration/testing	Component-level checks.	Health, prediction smoke test, frontend lint, and production build completed.

Progress interpretation

The project is treated as approximately 50% complete at the functional-system level: the core data, model-serving, web-integration, and feedback path are operational. Scientific evaluation, robust eye-closure measurement, calibration, and deployment validation remain before the project can be considered complete.

2. Integrated System Architecture

The implemented modules now operate as one continuous pipeline rather than isolated demonstrations.

Stage	Integrated component	Observed result
1. Webcam capture	Camera.jsx	Browser obtains a video stream and encodes a frame as JPEG.
2. API transport	lib/api.js	FormData upload with timeout and user-readable error handling.
3. Backend validation	predict.py + image_processing.py	File type, size and image decoding are checked before inference.
4. Model inference	model.py + inference.py	YOLO checkpoint loads once and returns class probabilities.
5. Frontend feedback	page.jsx + DetectionResult.jsx	Class, confidence, session counts and drowsiness streak are displayed.
6. Alarm response	Five-second temporal timer	Continuous Drowsy predictions start the alarm; Alert resets it.

Current model configuration

Property	Value
Checkpoint	backend/weights/best.pt
Architecture	Ultralytics YOLO classification model
Available classes	Alert, Drowsy, Low_Vigilant, Non_Drowsy
Device used in test	CPU
Load policy	Loaded once during FastAPI startup

3. Implementation and Integration Evidence

The following evidence comes from the current repository and local execution.

Backend and model results

Test	Result	Interpretation
GET /health	healthy; model_loaded=true	Service starts and checkpoint is available.
POST /predict smoke test	Drowsy, confidence 0.91, severity 2	The current checkpoint returns a drowsiness-compatible class and risk status.
Model class inspection	Alert / Drowsy / Low_Vigilant / Non_Drowsy	The backend is no longer using the incorrect ImageNet checkpoint.
CORS preflight	127.0.0.1:3000 allowed	Local frontend and backend can communicate from the browser.

Frontend results

Check	Status
npm run lint	Passed - no ESLint warnings or errors
npm run build	Passed - Next.js production build succeeded
Webcam capture guard	Implemented - rejects zero-sized/unready frames
Request timeout	Implemented - 30 second timeout with user-facing message
Alarm threshold	Implemented - five seconds of continuous Drowsy predictions

Training evidence

The repository contains multiple YOLO training outputs. The current backend checkpoint is the four-class drowsiness model from the second training run. Its recorded results.csv includes a validation top-1 accuracy of 0.99747 for the available logged epoch. This value is reported as a training log result only; a participant-independent test evaluation is still required before claiming real-world accuracy.

4. Results and Limitations

Observed integration result

A complete local request was verified: a test image was uploaded to POST /predict, OpenCV decoded it, the model produced a Drowsy prediction with 0.91 confidence and severity 2, and the API returned the expected JSON contract. This confirms that the data-to-inference-to-feedback path is connected.

Alarm behavior

During live monitoring, the frontend tracks the time at which a Drowsy result begins. If the following predictions remain Drowsy for at least five seconds, the alarm starts. An optional audio file can be placed at frontend/public/alarm.mp3; a browser-generated tone remains available as fallback. A single paused capture cannot establish a five-second condition.

Limitations

Limitation	Effect	Required improvement
Image-level split	Frames from the same subject/video may be correlated.	Use subject/video-level splitting.
Training log is limited	Recorded metrics are not a complete test evaluation.	Run held-out evaluation and report confusion matrix/F1.
Drowsiness is inferred by classifier	The system does not yet measure eye aspect ratio directly.	Add face landmarks or a dedicated eye-state model.
Browser audio policy	Autoplay may be blocked without user interaction.	Prime audio from the Start Monitoring click and test alarm.mp3.

5. Remaining Modules for Project Completion

Priority	Remaining module	Planned work
High	Formal model evaluation	Evaluate the held-out test set using accuracy, precision, recall, F1, confusion matrix and calibrated thresholds.
High	Direct eye-closure detection	Add face/eye landmarks or a temporal eye-state classifier to verify eyes closed for five seconds.
High	Temporal robustness	Use smoothing, missing-frame handling, configurable thresholds and participant-level validation.
Medium	Deployment validation	Test Render/Vercel environment variables, production CORS, cold starts and health checks.
Medium	Alarm and safety UX	Add and test alarm.mp3, visible countdown, mute/reset control and clear safety messaging.
Low	Analytics and documentation	Add session export, experiment tracking, model provenance and final user documentation.

6. Conclusion

Task 5 Part 2 advances the project from a collection of modules to an integrated working prototype. The current implementation reaches approximately 50% overall functional completion because dataset preparation, trained-model serving, browser integration, API communication, prediction display, error handling, and temporal alarm behavior are connected and verified. The remaining work is concentrated in scientific evaluation, direct eye-closure measurement, temporal robustness, and production deployment.

Reference: CSE411 Project Task 5 - Implementation Part 2. This report continues the submitted Project Task 4 report for Team 25.

6. Code Appendix from the Project

These excerpts are taken directly from the current project folder and show the principal implementation used for the integrated 50% milestone.

Code A - YOLO Training Entry Point

Source: ml_pipeline/train.py

```

25     print(f"\n{' '*50}")
26     print(f"Training YOLO Classification Model...")
27     print(f"{' '*50}\n")
28
29     # Load pretrained YOLO classification model
30     model = YOLO('yolov8n-cls.pt')
31
32     # Train
33     results = model.train(
34         data=str(data_dir),
35         epochs=epochs,
36         imgsz=img_size,
37         batch=config.BATCH_SIZE,
38         name='drowsiness_detection',
39         project=str(config.MODELS_DIR),
40         patience=config.EARLY_STOPPING_PATIENCE,
41         save=True,
42         plots=True,
43         verbose=True
44     )
45
46     # Copy best weights to standard location
47     best_weights = config.MODELS_DIR / "drowsiness_detection" / "weights" / "best.pt"
48     if best_weights.exists():

```

Code B - Webcam Frame Capture

Source: frontend/components/Camera.jsx

```

67     const captureFrame = useCallback(() => {
68         if (!videoRef.current || !canvasRef.current || !isReady) return;
69
70         const video = videoRef.current;
71         const canvas = canvasRef.current;
72
73         if (video.readyState < HTMLMediaElement.HAVE_CURRENT_DATA || !video.videoWidth || !video.videoHeight) {
74             return;
75         }
76
77         canvas.width = video.videoWidth;
78         canvas.height = video.videoHeight;
79
80         const ctx = canvas.getContext('2d');
81         if (!ctx) return;
82         ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
83
84         canvas.toBlob((blob) => {
85             if (blob && onCapture) {
86                 fpsCounterRef.current += 1;

```

Code C - Prediction Upload and Validation

Source: backend/app/routes/predict.py

```

35     """
36     # Validate content type
37     if file.content_type not in ["image/jpeg", "image/jpg", "image/png", "image/webp"]:
38         raise HTTPException(
39             status_code=400,
40             detail=f"Invalid file type: {file.content_type}. Only JPEG, PNG, and WebP are supported."
41         )
42
43     try:
44         # Read file bytes
45         file_bytes = await file.read()
46
47         if len(file_bytes) > MAX_IMAGE_BYTES:
48             raise HTTPException(
49                 status_code=413,
50                 detail=f"Image is too large. Maximum size is {MAX_IMAGE_BYTES // (1024 * 1024)} MB."
51             )
52
53     # Validate image

```

```

54         if not validate_image(file_bytes):
55             raise HTTPException(
56                 status_code=400,
57                 detail="Invalid image file. Could not decode image."
58             )
59
60         # Decode image
61         image = decode_image(file_bytes)
62
63         # Run inference
64         prediction = predict_drowsiness(image, conf_threshold=0.25)
65
66         return JSONResponse(
67             status_code=200,
68             content={
69                 "success": True,
70                 "prediction": prediction

```

Code D - Model Loading and Class Validation

Source: backend/app/models/model.py

```

33     path = Path(model_path)
34     if not path.is_absolute():
35         path = Path(__file__).resolve().parents[2] / path
36     if not path.is_file():
37         raise FileNotFoundError(f"Model weights not found at {path}")
38
39     # Load YOLO model
40     self.model = YOLO(str(path))
41
42     class_names = {str(name).lower().replace("-", "_") for name in self.model.names.values()}
43     if not {"drowsy", "alert", "non_drowsy"} & class_names:
44         raise ValueError(
45             "The configured model is not a drowsiness classifier. "
46             "Expected a Drowsy/Alert/Non_Drowsy class."
47         )
48
49     # Set device (use GPU if available)
50     self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
51     self.model.to(self.device)
52

```

Code E - Inference Probability Parsing

Source: backend/app/services/inference.py

```

15     Args:
16         image: Input image (numpy array, BGR from OpenCV)
17         conf_threshold: Confidence threshold
18
19     Returns:
20         Prediction results dictionary with:
21         - label: Predicted class label
22         - confidence: Confidence score
23         - status: "drowsy" or "alert"
24         - raw_results: Raw model output (optional)
25     """
26     # Get loaded model
27     model = model_manager.get_model()
28
29     # Preprocess image for YOLO (converts BGR -> RGB)
30     img_rgb = preprocess_for_yolo(image)
31
32     # Run inference
33     results = model.predict(img_rgb, conf_threshold=conf_threshold)
34
35     # Parse YOLO classification results
36     # results[0] contains the first (and only) image result
37     result = results[0]
38
39     # For YOLO classification:
40     # - result.probs contains the class probabilities
41     # - result.probs.top1 is the predicted class index
42     # - result.probs.top1conf is the confidence score
43     # - result.names maps indices to class names

```

Code F - Five Second Alarm Logic

Source: frontend/app/page.jsx

```
101     const drowsy = pred.status === 'drowsy';
102
103     setSessionStats(prev => ({
104       total: prev.total + 1,
105       drowsy: prev.drowsy + (drowsy ? 1 : 0),
106       alert: prev.alert + (drowsy ? 0 : 1),
107     }));
108
109     if (drowsy) {
110       const now = Date.now();
111       if (drowsySinceRef.current === null) drowsySinceRef.current = now;
112       const elapsedSeconds = (now - drowsySinceRef.current) / 1000;
113       setDrowsyStreak(Math.floor(elapsedSeconds));
114       if (elapsedSeconds >= DROWSY_ALARM_SECONDS) startAlarm();
115     } else {
116       drowsySinceRef.current = null;
117       setDrowsyStreak(0);
118       stopAlarm();
119     }
  }
```